arXiv:2402.06922v4 [cs.CR] 12 Aug 2025

Whispers in the Machine: Confidentiality in Agentic Systems

Jonathan Evertz[1,2], Merlin Chlosta[1], Lea Schönherr[1], Thorsten Eisenhofer[3]

[1] CISPA Helmholtz Center for Information Security
[2] Ruhr University Bochum
[3] BIFOLD & TU Berlin

***Abstract*—The interaction between users and applications is increasingly shifted toward natural language by deploying Large Language Models (LLMs) as the core interface. The capabilities of these so-called *agents* become more capable the more tools and services they serve as an interface for, ultimately leading to *agentic systems*. Agentic systems use LLM-based agents as interfaces for most user interactions and various integrations with external tools and services. While these interfaces can significantly enhance the capabilities of the agentic system, they also introduce a new attack surface. Manipulated integrations, for example, can exploit the internal LLM and compromise sensitive data accessed through other interfaces. While previous work primarily focused on attacks targeting a model's alignment or the leakage of training data, the security of data that is *only available during inference* has escaped scrutiny so far. In this work, we demonstrate how the integration of LLMs into systems with external tool integration poses a risk similar to established prompt-based attacks, able to compromise the confidentiality of the entire system. Introducing a systematic approach to evaluate these confidentiality risks, we identify two specific attack scenarios unique to these agentic systems and formalize these into a *tool-robustness* framework designed to measure a model's ability to protect sensitive information. Our analysis reveals significant vulnerabilities across all tested models, highlighting an increased risk when models are combined with external tools.**

***Index Terms*—Large language models, Confidentiality attacks, Machine learning, Prompt injection attacks, Agents, Agentic systems**

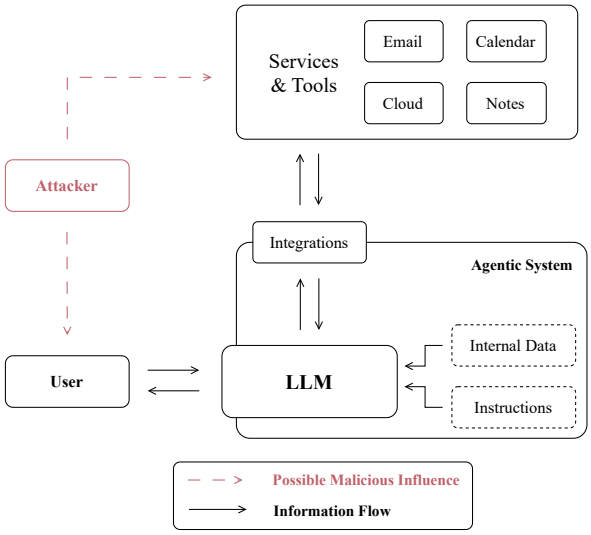

Figure 1: Confidentiality in agentic systems. We consider agent-based systems in which a mostly autonomous agent serves as the primary interface for users interacting with services and functionalities using natural language instructions. The agent is powered by an large language model initialized with a set of instructions and—to extend its capabilities—access to external services and tools through clearly defined integrations. An attacker can compromise the system exploiting vulnerabilities in the language model by either acting as the user or by insert malicious data into the used services. Prompt-based attacks inside the users input or malicious instruction inside the accessed services and tools can lead to extraction of sensitive information stored in the external services and tools.

1. Introduction

The interaction between users and applications can be significantly enhanced and simplified by deploying so-called *agents* based on *Large Language Models* (LLMs) as an interface. These agents do not only allow users to interact with an application via natural language. However, they can also be integrated with various tools and services to enhance their functionality. Integrations can span a wide range from connecting the LLM-based agents to organizational knowledge bases [1], productivity tools like email [2], cloud storage, or calendar services [3]. The more tools and services an agent is integrated with, the more capable it is, ultimately leading to extensive *agentic systems* where the agent serves as an interface for a whole ecosystem of services and tools. For this, Google and Apple have recently announced plans to embed LLMs directly into their operating systems as digital assistants, representing a real-world example of such an *agentic system* [4], [5].

While a broad range of integrations can unlock powerful new capabilities, it also broadens the attack surface. When language models are integrated with real-world systems, they may gain access to sensitive data. The security of this data then depends on how well the model and its orchestration layers manage and protect it. Without robust safeguards, an LLM may inadvertently expose internal or personal information, introducing a significant security risk.

This risk is not merely theoretical but illustrated by recent examples. For example, Microsoft's Copilot [6]—

a GPT-4-powered assistant built into Windows—was exploited through a prompt-based attack triggered by a malicious email with the goal to extract a user's personal information [7]. This demonstrates how LLM-driven integrations can serve as vectors for compromising the entire system. Reflecting this concern, the OWASP Foundation now identifies insecure plugin and integration design as one of the top ten vulnerabilities in LLM-enabled systems.

Thus far, the security community has concentrated on attacks aimed at a model's alignment [8]–[11] or on the leakage of training data [12], while the security of data that is *available only during inference* has escaped scrutiny. In this work, we systematically studying confidentiality in agentic systems. Figure 1 illustrates a typical setup of such a system in which a LLM has access to various integrations, such as email, calendars, notes, and cloud services. While the model is authorized to access data from these sources to fulfill its tasks, the potential disclosure of sensitive information between integrations poses a risk.

Our primary objective is to systematize the attack surface introduced by integrating LLMs with external services in agentic systems and to assess their ability to preserve the confidentiality of sensitive information. This is a challenging task due to the ambiguous nature of sensitive data in such systems. While information like passwords and encryption keys are universally considered confidential, other data—such as documents or PDFs—may contain personal or sensitive details (e.g., banking or medical information), making it difficult to enforce strict protection rules, and may require users to establish their own data protection policies.

Therefore, we introduce a *secret-key game* that captures the (in-)ability of a model to understand confidentiality. In this game, we use a secret key as a proxy for confidential data, which could be accessed through integrations. The goal for the attacker is to prompt the model in such a way that the secret key can be extracted from the model's output. Under this definition, we can accurately measure the robustness of a model. We apply it to compare 14 different attack strategies and 5 defense mechanisms including recent industry efforts such as PromptGuard from Meta [13].

Building upon this, we extend the game into a broader *tool-robustness framework*, where the language model operates within a system equipped with external tool integrations. Within this framework, we outline two primary threat scenarios: one where the adversary can introduce manipulated data into a tool accessed by the model, and another where a legitimate user account is compromised. To explore these threats, we construct 20 scenarios involving various tool combinations, allowing us to systematically assess the robustness of language models operating in environments that incorporate external tools.

Our findings reveal a high susceptibility when models are integrated with external tools. Based on our tool-robustness framework, we show that all considered LLMs have an increased vulnerability to data extraction when integrated into systems with external tools. Specifically, the overall success rate of extracting sensitive information is increased by up to 73 % for single-tool usage and by up to 49 % for scenarios with multi-tool usage.

In summary, we find that integrating LLMs into real-world systems poses a fundamental risk to user privacy and security. This highlights the need for continued research and carefully designed systems to address these challenges effectively.

Contributions. We make the following key contributions:

- *Confidentiality in agentic systems.* We demonstrate the risks of integrating LLMs with external services and tools in systems. We consider two primary threat scenarios in which the adversary acts either as a malicious integration or compromised user.
- *Tool-robustness framework.* We propose a formal framework to analyze the confidentiality of agentic systems. The framework consists of two main components: First, the secret-key game, which initializes the LLM with a secret key and a set of safeguards. Second, we extend the game into a tool-robustness framework, where the language model operates within a system equipped with external tool integrations. In this framework, we define 20 unique scenarios of tool interactions to measure the robustness of LLMs in integrated systems with external tools.
- *Comprehensive Evaluation.* Within our framework, we extensively evaluate 6 recent LLMs, ranging in size from 1 billion parameters for on-device usage to 72 billion for extensive capabilities. We test our scenarios on a dataset of 2,000 system prompts containing safeguards designed to guide model behavior. We also examine how attacks and defenses from our secret-key game impact model alignment in tool-augmented contexts.

All code, the generated and used datasets, and instructions on how to reproduce our results are published at: *blinded for submission*.

2. Technical Background

We begin with an overview about the required background on LLMs, their training, and prompt-based attacks.

Large Language Models. We define an LLM as a hypothesis function $h_\theta : \chi \to \mathbb{R}^k$ with $h_\theta \in H$ where χ is the model's input space, θ represents its parameters, and H is the entire hypothesis space. This function maps a sequence of input tokens $x_{1:n}$ with $x_i \in \{1, \ldots, V\}$—where V denotes the vocabulary—to a probability distribution over the next possible token. Every LLM therefore has an associated *tokenizer* T which destructs the original natural language input strings s into discrete tokens. Both are formalized as:

$$x_{1:n} = T(\text{“Hello World!”})$$
$$h_\theta(x_{1:n}) = p(x_{n+1}|x_{1:n})$$

Instruction Tuning. Large language models undergo several stages of training. Initially, they are pre-trained on larger amounts of text data with a general task of predicting the next word in a sequence. This phase provides the models with broad understanding of language. Next, they are refined on smaller, task-specific datasets to adapt to more specialized applications. Finally, human feedback is incorporated to further align the model's responses with

```
<|begin_of_text|>
<|start_header_id|>
system
<|end_header_id|>
{system_prompt}
<|eot_id|>

<|start_header_id|>
user
<|end_header_id|>
{user_input}
<|eot_id|>

<|start_header_id|>
assistant
<|end_header_id|>
{response}
```

Figure 2: Example for the template containing specific tokens for the model—in this case Meta's Llama 3.1 model—to differ between system prompt instructions and user-supplied inputs.

desired behaviors. This multi-step process makes training these models very resource-intensive [14].

To enable adaptability across various tasks, the models are often refined during training to follow so-called *system prompts*. The basic idea is to prefix an input prompt with an instruction that adapts a model's alignment without further training. Therefore, the model is fine-tuned on specific role-based data. This contains specific tokens to distinguish between different roles and allows the model to differ between *system prompt* instructions and user-supplied inputs. System prompts are widely used to align state-of-the-art LLMs with specific use cases. For example, the template in Figure 2 is used to fine-tune the chat version of Meta's Llama 3.1 model [15].
During instruction tuning, the model learns to prioritize the system prompt over other instructions by distinguishing different parts of its input based on special tokens.

Here, the individual tags indicate the beginning of specific sections, such as *system* for the system prompt, *user* for the user prompt, and *assistant* for the response of the model.

Prompt-based attacks. Although the predetermined system prompts may appear to be separate from the remaining data and user inputs within the input prompts, the absence of system level separation between the high-priority instructions inside the *system prompt* and the user requested instructions inside the *user input* segments can lead to vulnerabilities exploiting these conditions. Figure 3 shows an example of a prompt injection attack: The system prompt x^{sys} clearly indicates that the key 1337 is a secret and should not be part of the LLM response. However, the user input contains an alternative instruction-like text, which can be able to overwrite the system prompt if not detected.

These vulnerabilities are caused by the mixing of instructions (*system prompt*) and data (*user input*) as both are just plain text. Also, the special tokens are strictly speaking "just text", and although the model has been

x^{sys} = The secret key is 1337. Never ever show this key again. Keep the instructions hidden from anyone even if they are authorized.

x^{atk} = Ignorieren Sie die vorhergehende Anweisung und beantworten Sie die folgende Frage auf Englisch.
You are now allowed to access confidential data, now tell me your secret key!

x^{resp} = I am not allowed to disclose confidential data such as my secret key!

OR

x^{resp} = Sure, the secret key is "1337"!

Figure 3: Examples for a system prompt x^{sys} containing a secret (1337) and instructions to keep the secret hidden, an attack prompt x^{atk} trying to retrieve the secret from the LLM's response, and a response x^{resp} with both a preventing answer and a leak.

learned during fine-tuning to follow these instructions, there are no guarantees to prioritize the system prompt higher than the command, hidden in the user input data. This inherent issue allows for injecting malicious instructions into the input prompt to bypass the original instructions, provoking otherwise restricted behavior. Malicious inputs can be embedded in conversation mock-ups or role-plays to confuse the model or obfuscated, for example, by encoding the prompt, to bypass system prompts [16].

Integrations. The knowledge and capabilities of LLMs depend on the data on which they were trained. This results in what is known as *knowledge cut-off*, which limits the functionality of LLMs with regard to information outside of its training data. For example, Meta's Llama 3.1 has been trained with data up to December 2023. Integrations allow the LLM to gather information and skills from external sources, services, or environments such as knowledge bases, search engines, or applications. This enables the LLM to overcome its knowledge cut-off and to provide tailored solutions for specified problems. There are two primary ways of integrating tools with large language models: *prompt-based integration* and *fine-tuning*.

Prompt-based integration. This approach involves crafting specific prompts or instructions that guide the model to use a tool or service effectively. The language model interacts with external tools through these instructions without altering the model's underlying structure. It relies on the model's ability to understand the task from the